Introduction to Artificial Intelligence

Chapter 6 - Game Trees

Namyup Kim

Soongsil University



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

Recap of Chapter 5

- A constraint satisfaction problem is
  - Variables, Domains and Constraints
  - Its goal test (*constraint*) is something an algorithm can read during searching.
- Backtracking search is depth-first with one variable at a time and the constraints checked as you go
- Filtering and ordering are what make it fast ; min-conflicts often beats all of it.
- But every problem until ***Ch.5*** had a single agent in it.



*Today there are **Two Agents**, and the second one is trying to make you lose.*



Contents

- Adversarial games: what a game is, written down formally, and what zero-sum buys you
- Adversarial search: the value of a state when somebody else controls half the moves, and the minimax algorithm
- Game tree pruning: alpha-beta, which gives the same answer for far less work
- Resource limits and evaluation functions: what to do when you cannot reach the end of the game
- Uncertain outcomes: "*expectimax*", where the opponent is replaced by an average
- Utilities and rationality — what the numbers we average actually are, and whether we are allowed to average them

Learning Objectives

- **Write a game down formally**: states, players, actions, terminal test, terminal utilities and say what the zero-sum assumption buys you
- Compute **minimax values** by hand, and run **alpha-beta pruning** on a tree, including saying what it does and does not tell you afterwards
- Explain why a depth limit forces you to invent an **evaluation function**, and design one for a given game
- Choose between minimax and **expectimax** by looking at the environment, and say what each assumption costs when it turns out to be wrong
- State what a utility is, why averaging needs more than an ordering, and what the axioms of rationality buy you

Introduction to Artificial Intelligence

CS188 6 - Part I: Adversarial Search

Namyup Kim

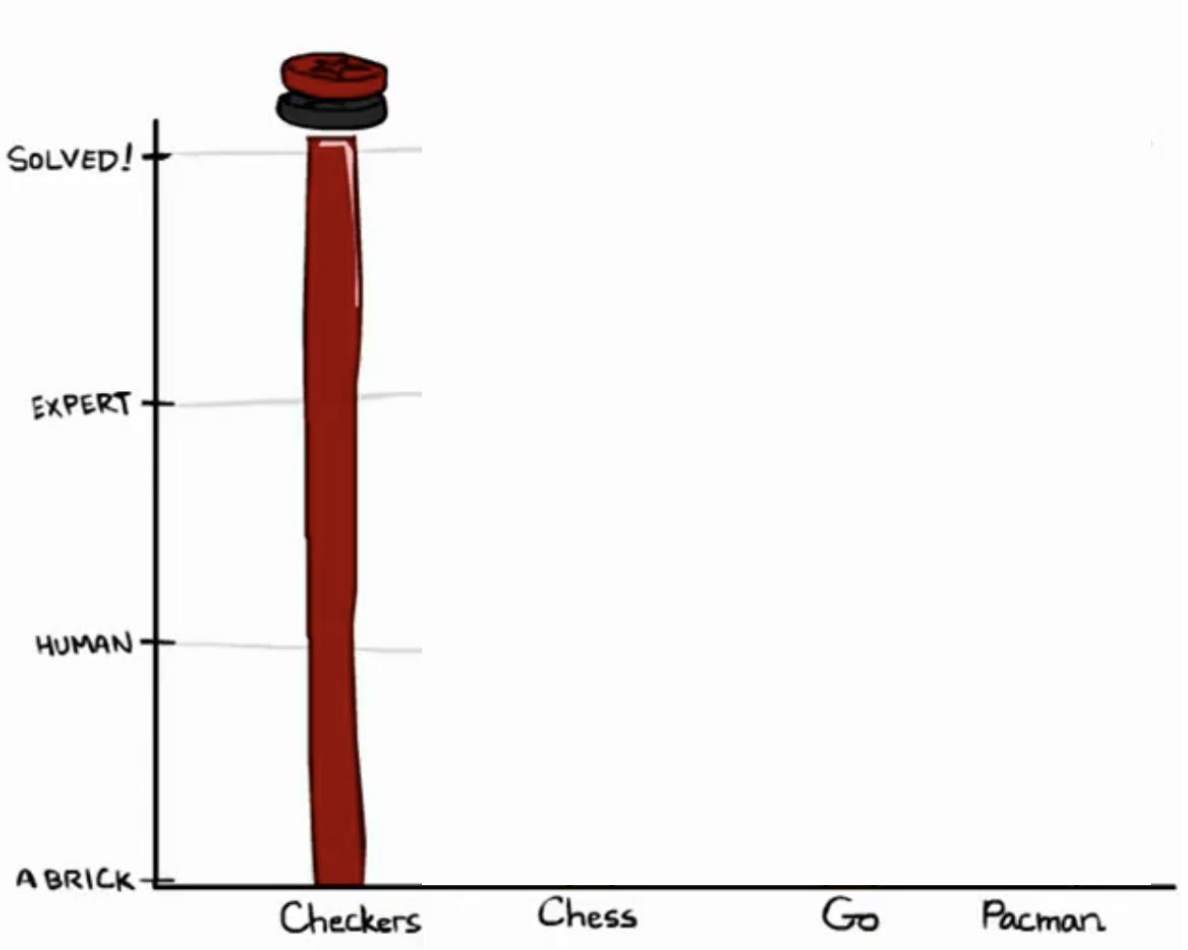
Soongsil University



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

Game Playing State-of-the-Art

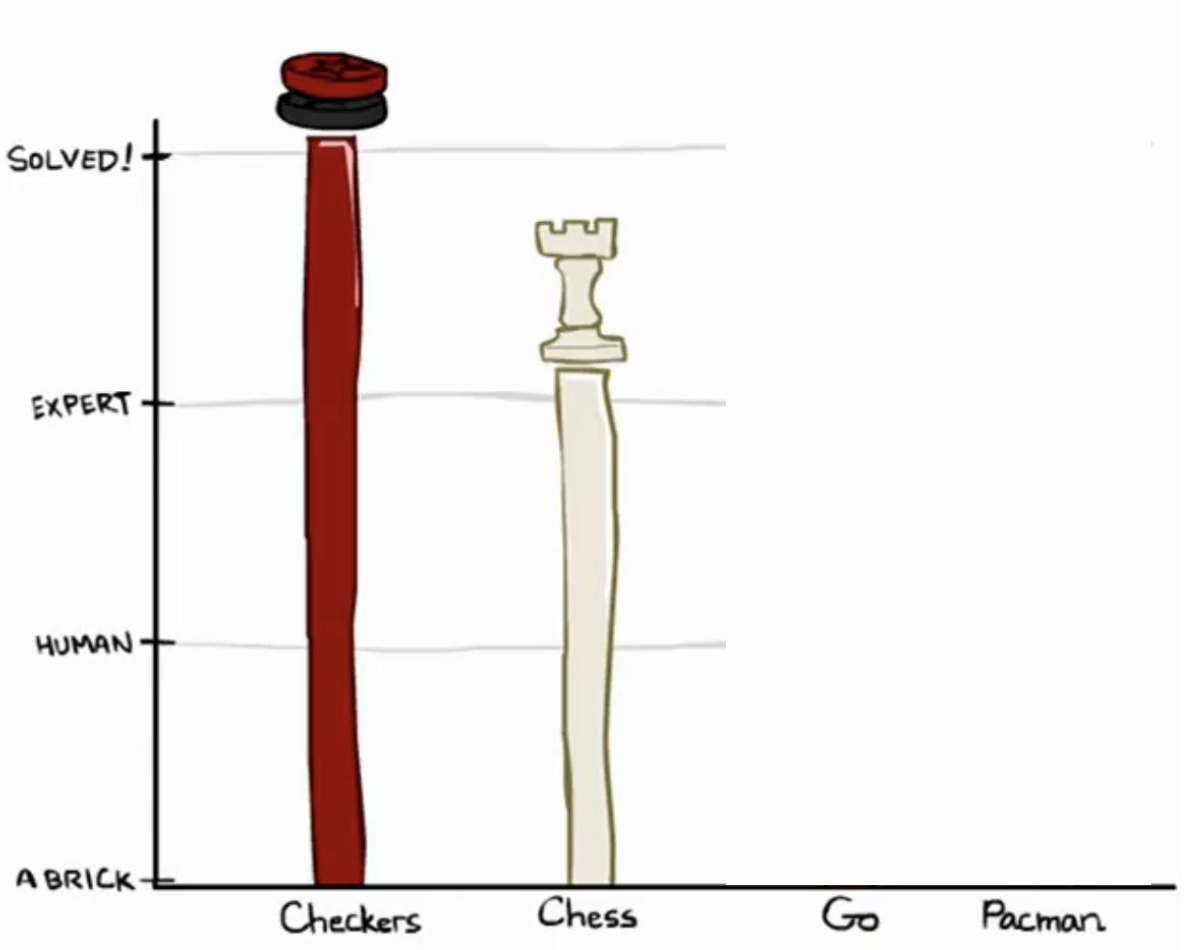
- CHECKERS:
  - 1950: the first computer player.
  - 1994: Chinook ends Marion Tinsley's forty-year dynasty as human champion.
  - 2007: checkers is SOLVED
  - SOLVED means **we know the whole game tree**. Perfect play from the start is a draw.



Game Playing State-of-the-Art

- CHECKERS:
  - 1950: the first computer player.
  - 1994: Chinook ends Marion Tinsley's forty-year dynasty as human champion.
  - 2007: checkers is SOLVED
  - **SOLVED means we know the whole game tree**. Perfect play from the start is a draw.

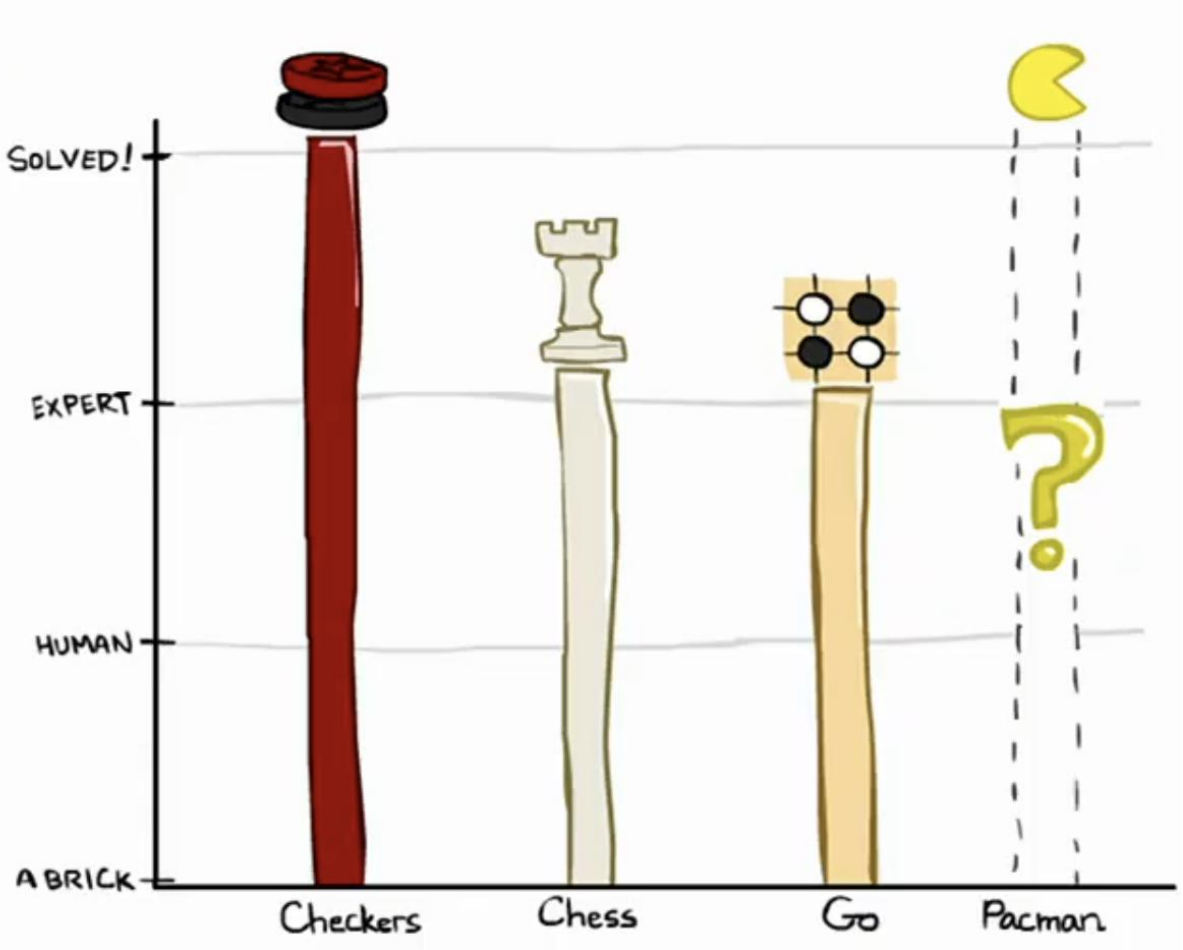
- CHESS:
  - 1997: **Deep Blue** beats Garry Kasparov in a six-game match. It looked at 200 million positions per second.
  - It used a strong evaluation function and extended some lines of search as deep as 40 ply.



Game Playing State-of-the-Art (2)

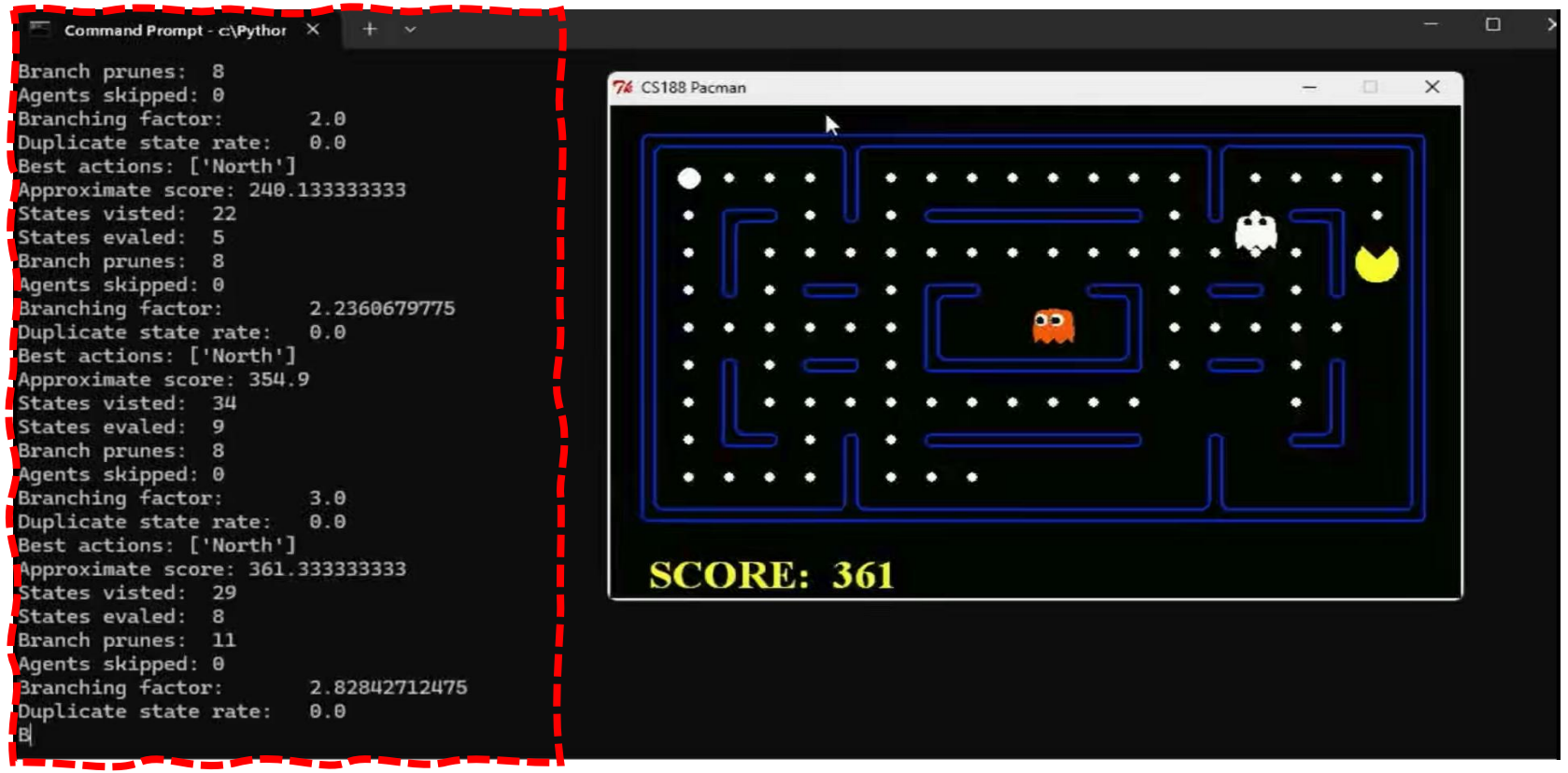
- GO: 2016, AlphaGo beats the human champion. It uses Monte Carlo Tree Search together with a learned evaluation function
  - Go took nineteen more years than chess. The board is larger, the branching factor is about 250, and nobody could write a good evaluation function by hand
  - That gap is the whole story of this chapter: searching is not enough. You also need a good way to judge a position you cannot finish

- And PAC-MAN: the running example for this course



Behavior from Computation

- A good choice for behavior: (1) Memory (learning; history) and (2) Computation (simulation; future)
- A Pac-Man demo. Watch the computation, not the game.
- Nobody wrote those behaviors. They come out of searching a tree
- One thing to notice: the scores on screen look like 1,442.47. No real score in this game can look like that. What are those numbers?



1. Adversarial Games

What changes when somebody else is trying to beat you

- The axes (dimensions; categories) along which games differ
  - A formal definition of **a deterministic game**
  - **Zero-sum games**, and why one number is enough
  - **Plans** vs **Policies**



Types of Games

- Many Different Kinds of Games

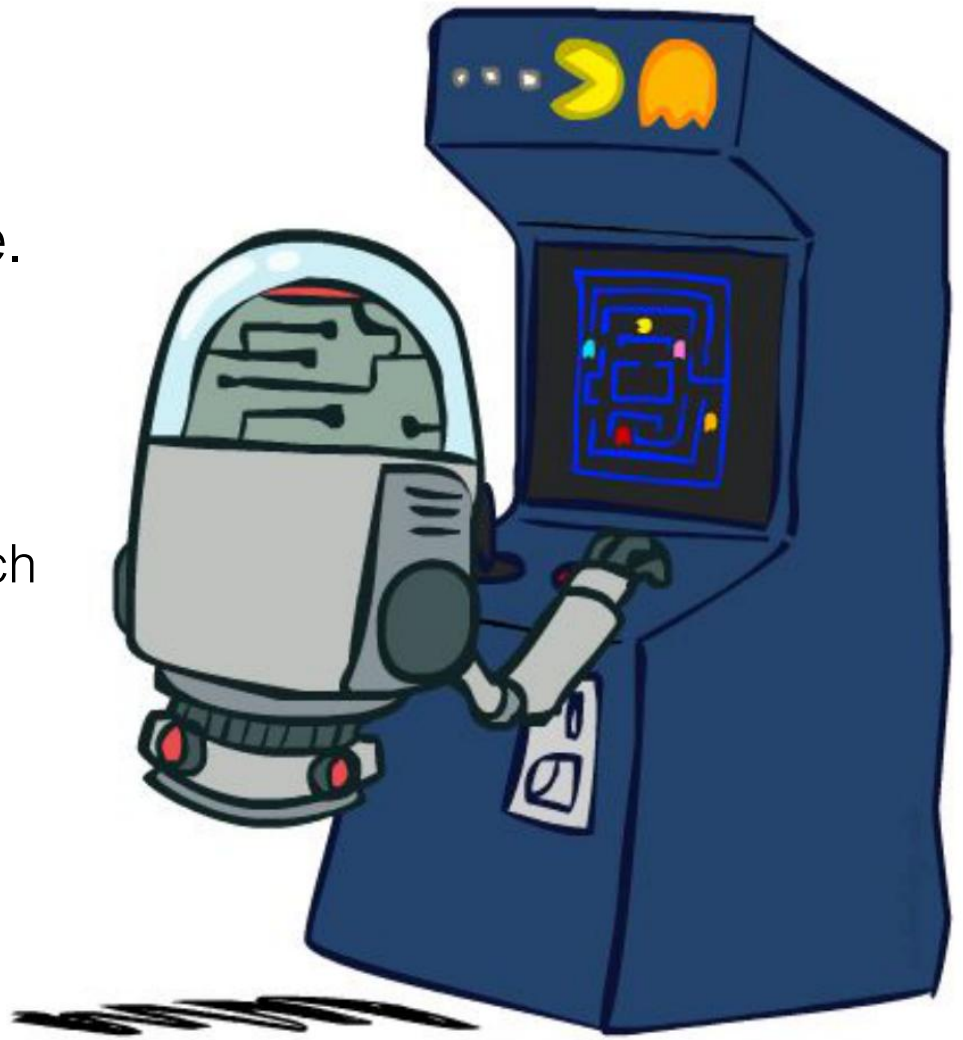


- Axes:
  1. **Deterministic or stochastic?**
     Chess is deterministic. Monopoly is not - you roll dice
  2. **One, two, or more players?**
     One player is just ordinary search
  3. **Zero sum?**
     If one side wins exactly what the other loses.
     Real life is usually not like this
  4. **Perfect information (can you see the state)?**
     Can you see the whole state, or is part of it hidden from you?

- We want algorithms that compute *a STRATEGY (a policy)* which recommends a move from each state, not a single plan

Deterministic Games

- Many possible formalizations, one is:
  - States: S, starting at s_0.
  - Players: $P = \{1 \ldots N\}$, usually taking turns
  - Actions: A (may depend on the player or the state.
  - Transition function: $S \times A \rightarrow S$
  - A terminal test : $S \rightarrow \{\text{True}, \text{False}\}$
  - Terminal Utilities: $S \times P \rightarrow R$
    - The utilities are the new part. In search, cost sat on each action. Here you play to the end and get one number.
- A solution is a *POLICY* $\pi: S \rightarrow A$



Zero-Sum Games



- Zero-Sum Games
  - The players have opposite utilities (values on outcomes). What one gains, the other loses.
  - Let's us think of a single value that one maximizes and the other minimizes.
  - Adversarial, pure competition.

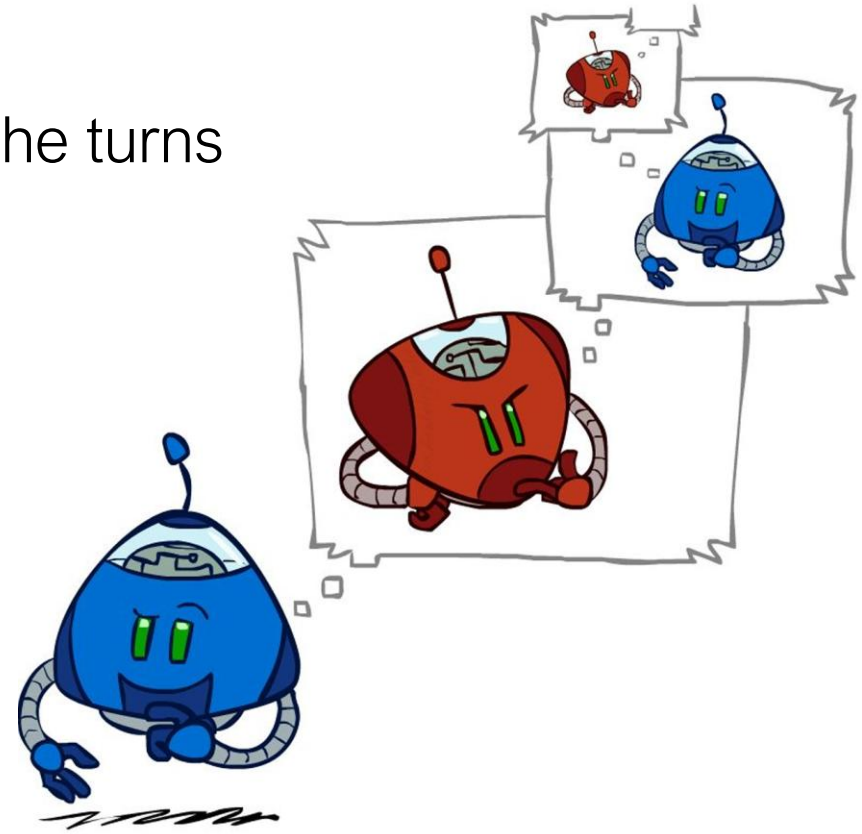


- General Games
  - Each player has an independent utility.
  - Cooperation, indifference and competition are all possible
  - Between the two extremes you get players who cooperate in some parts of the tree and compete in others

2. Adversarial Search

The value of a state, when somebody else controls half the moves

- Single-agent trees, and the value of a state
- What changes when an opponent takes some of the turns
- Minimax values, and the minimax algorithm
- How expensive it is, and why that is a problem



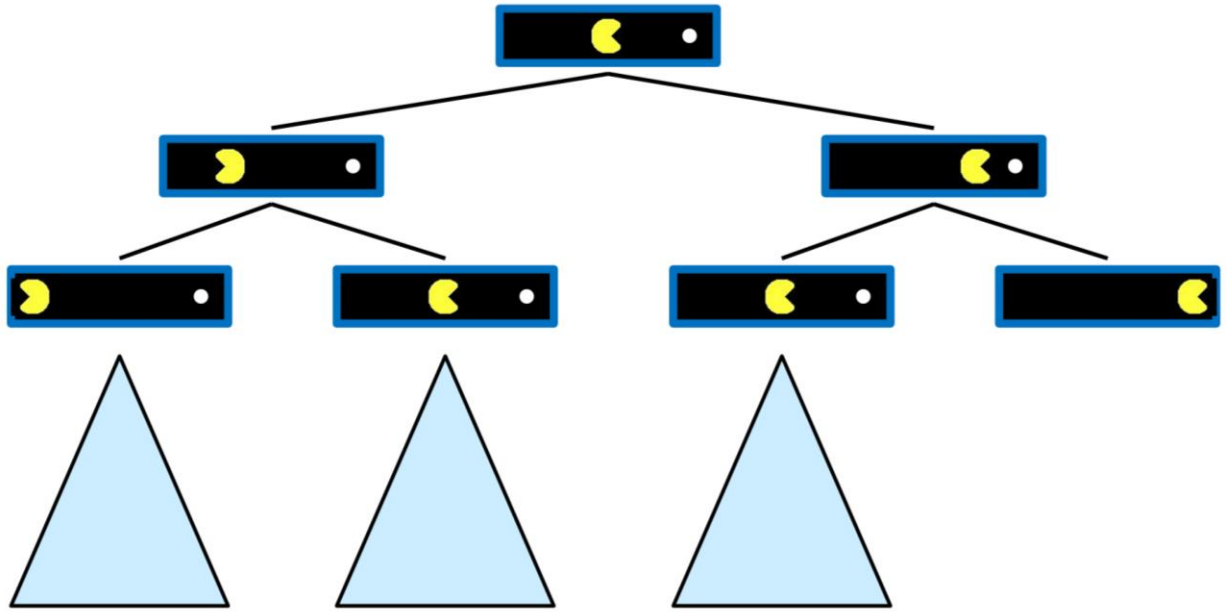
Single-Agent Trees

- Start with the easy case: only you are playing (***a single agent tree***)

- From the current state you have some actions. Each one leads to a new state, and from there more actions, and so on.

- This is exactly a search problem. You control every action, so **you can make any branch happen.**

- At the bottom, the numbers are utilities. A score of 8 here might be +10 for the dot and −1 for each of two steps.



Value of a State in Single-Agent Trees

- **Value of a state**: the best outcome (utility) you can reach from it
  → *the best achievable utility*

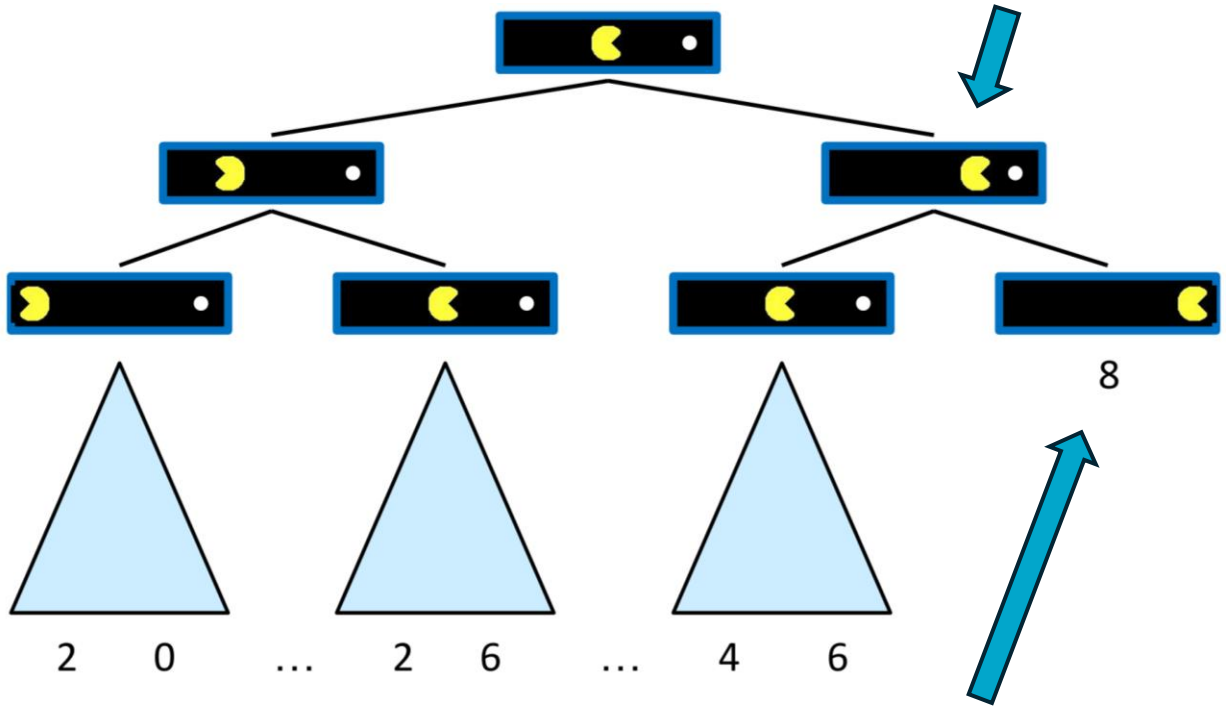
- (1) At a terminal state, the value is given. The game tells you.

- (2) At any other state, the value is the **MAXIMUM** of the values of its children.

- So, values flow upward from the bottom. The definition is ***recursive.***

(2) **Non-Terminal States:**

$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$



(1) **Terminal States:**

$$V(s) = \text{known}$$

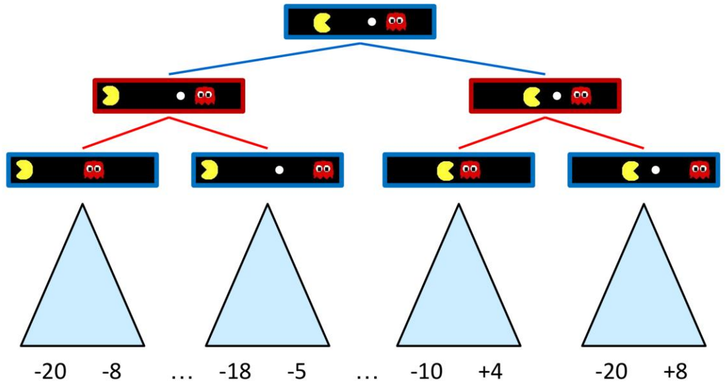
Adversarial Game Trees

- Now add an opponent. The tree holds the same set of futures as before

- The only thing that changed is CONTROL[2].
  - At some nodes you choose.
  - At others, the ghost chooses

- You do not know what the ghost will do. But you do know what it CAN do

- And in a zero-sum game you know what it WANTS: the opposite of what you want



2 **CONTROL(통제권)** — 트리는 그대로, 바뀐 건 **누가 고르느냐**뿐

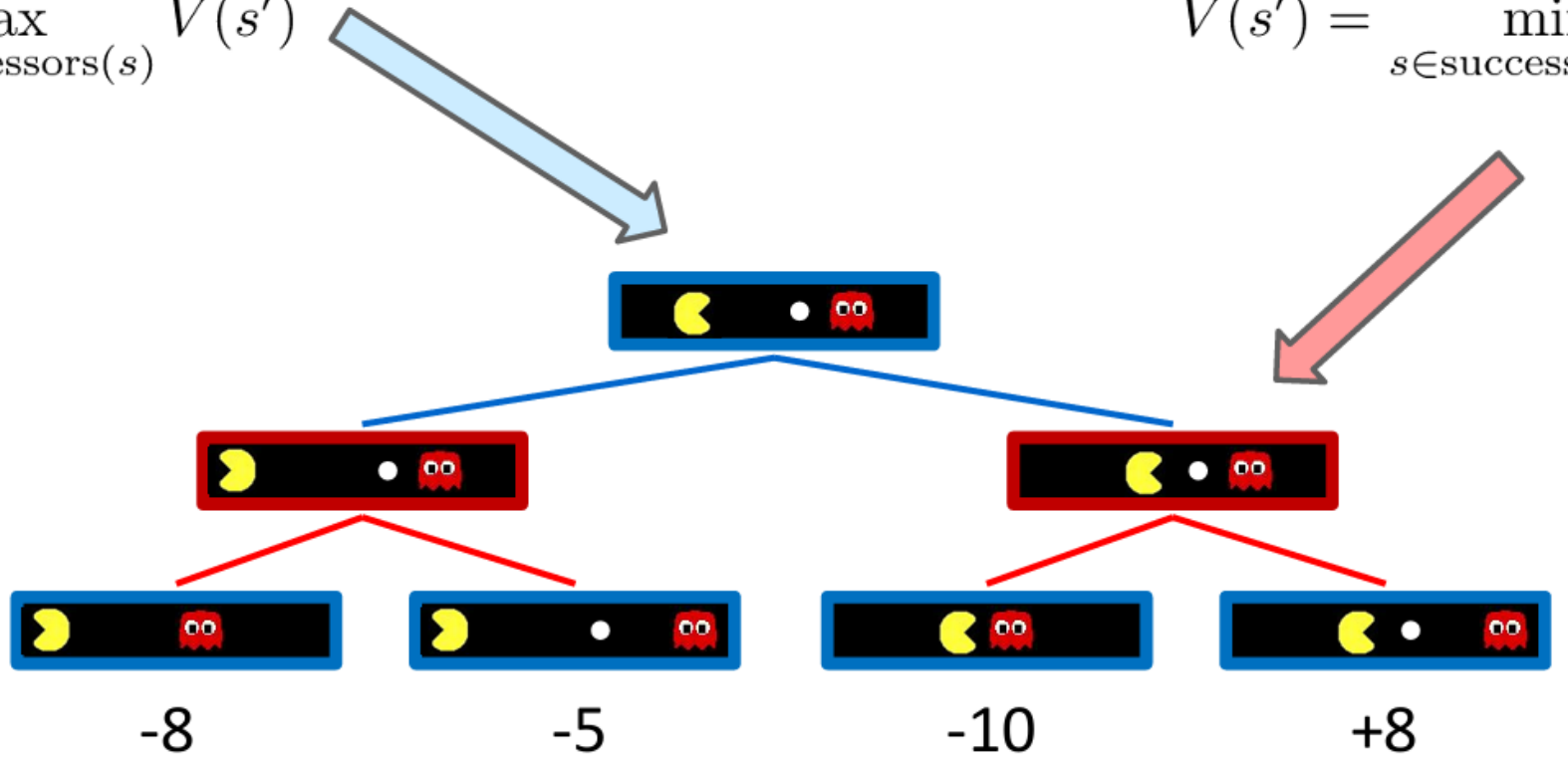
Minimax Values in Adversarial Game Trees

States Under Agent's Control:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

States Under Opponent's Control:

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

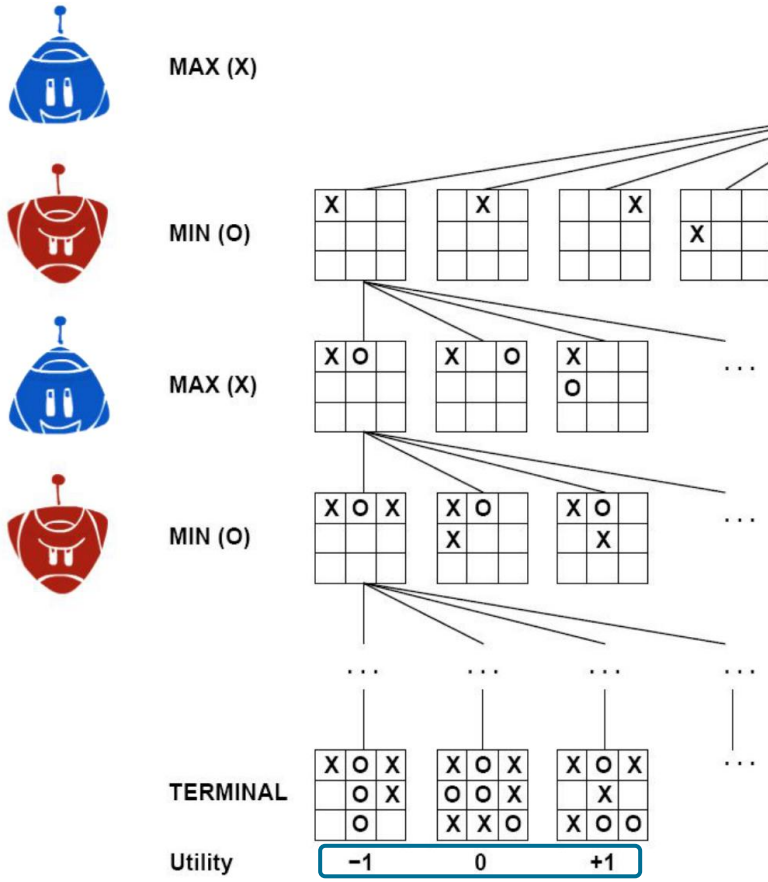


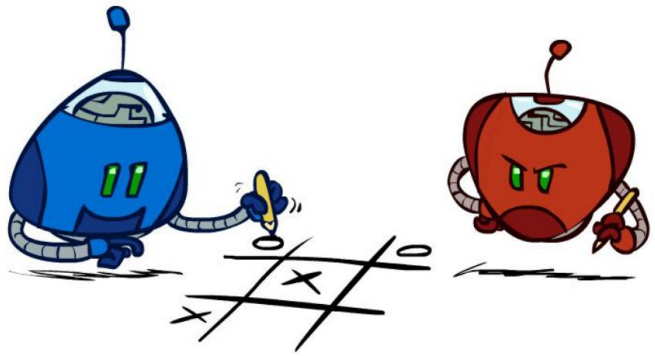
Terminal States:

$$V(s) = \text{known}$$

Example: Tic-Tac-Toe Game Tree

- The same idea on a game everybody knows.
- In tic-tac-toe, the terminal utilities are only +1, 0 and −1: I win, we draw, I lose.
- There is no averaging anywhere here. It is a $\max(\min(\max(\min(\max(\ldots)))))$
  → **The root's value must be one of the leaf values.**

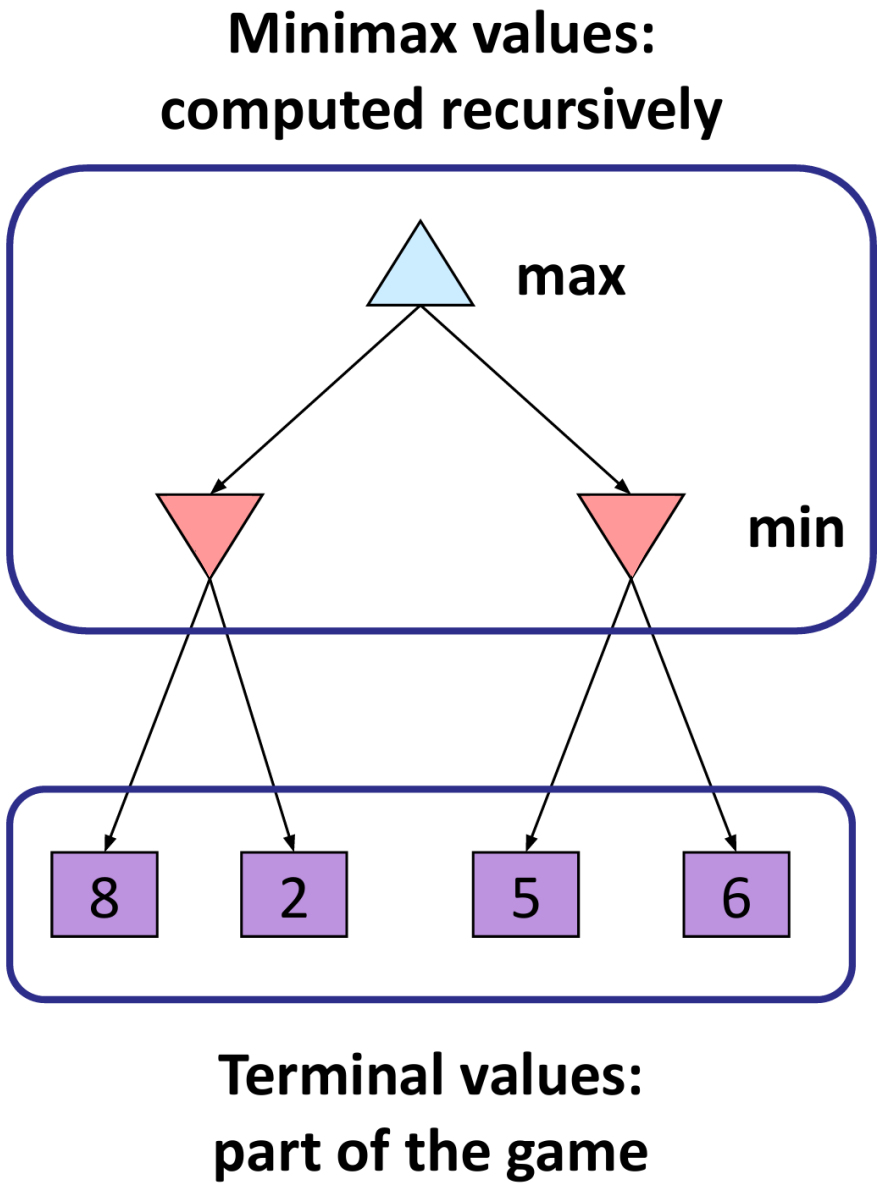




Adversarial Search (Minimax)

- The setting: deterministic, zero-sum games
  - Tic-tac-toe, chess and checkers
  - One player maximizes the result.
  - The other minimizes it

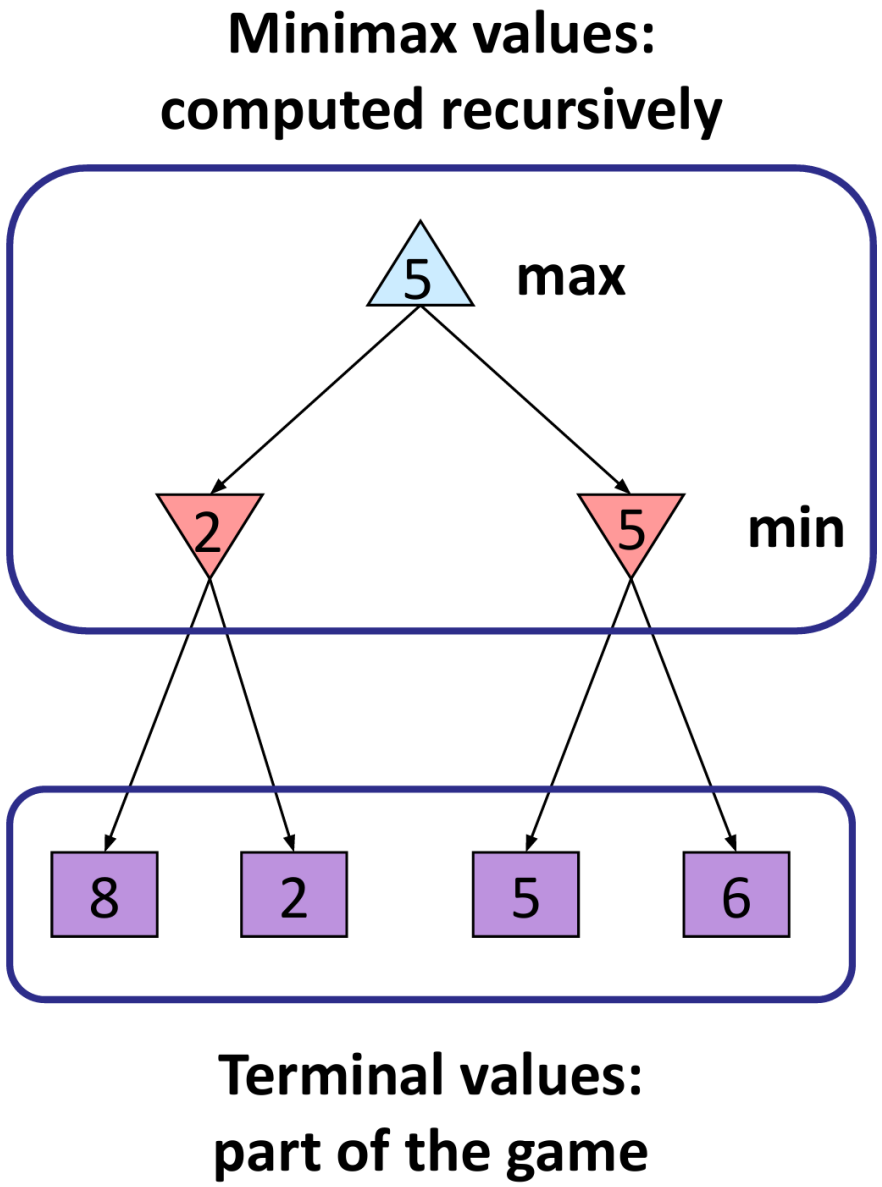
- Minimax search:
  - A state-space search tree
  - Players alternate turns.
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) opponent.



Adversarial Search (Minimax)

- The setting: deterministic, zero-sum games
  - Tic-tac-toe, chess and checkers
  - One player maximizes the result.
  - The other minimizes it

- Minimax search:
  - A state-space search tree
  - Players alternate turns.
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) opponent.



Minimax Implementation

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```



```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

- Two functions that call each other: a mutual recursion.[1]
- Watch the bounds move. In max-value, v starts at -inf and **RISES**. In min-value, v starts at +inf and **FALLS**. Alpha-beta needs this.

[1] max-value 안의 **min-value(successor)**, min-value 안의 **max-value(successor)** → 두 함수가 서로를 부르는 **상호 재귀(mutual recursion)**

Minimax Implementation (Dispatch)

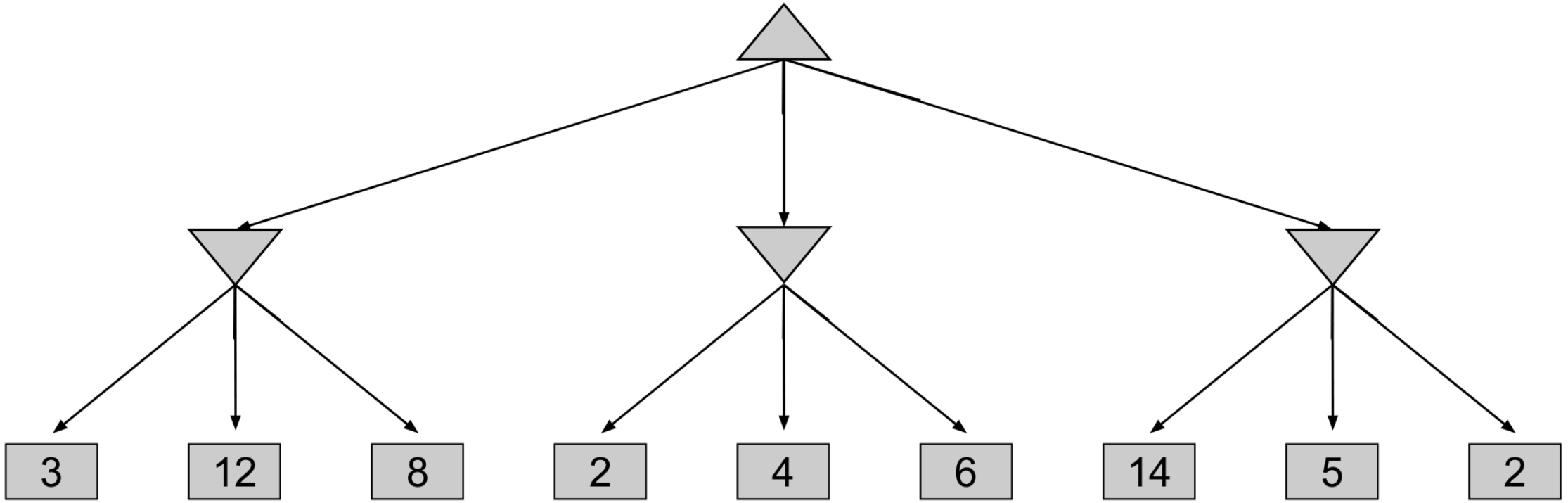
```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

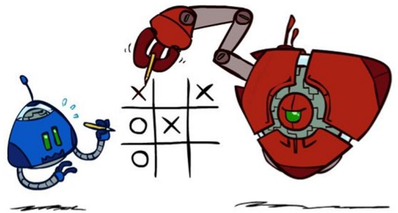
*Every state has a **value**. What decides it is who controls the state.*

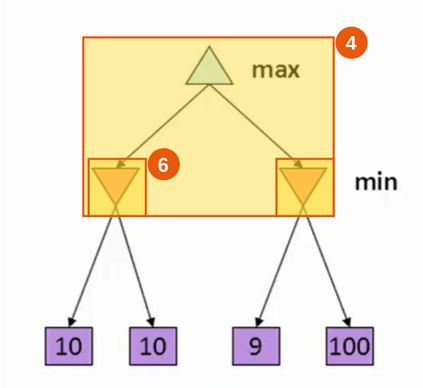
Minimax Example

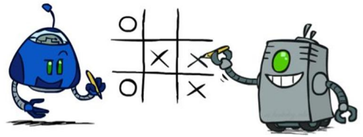


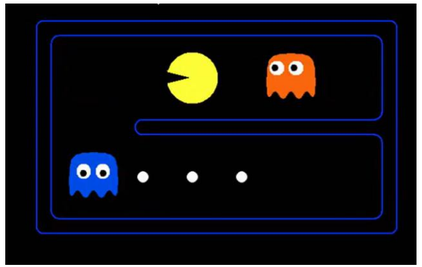
Minimax Properties

- Minimax is optimal against a perfect player. That is the guarantee
  - What if the opponent is not perfect[3]? Then you still do at least as well as the minimax value. Their mistakes can only help you
  - But you may do much worse than you could have 미내. Minimax will not take a risk that a perfect opponent would punish, even against an opponent who would not punish it
- In the demo: against a ghost that moves randomly, minimax Pac-Man throws itself at the ghost[5]









Optimal against a perfect player. Otherwise?

3 상대가 완벽하지 않을 때(not perfect) — 중요!

4 두 번째 그림(**트리 그래프**): **세모 칸(max △, min ▽) 안에 숫자(값)를 쓰라고 함**

5 게임 트리가 있을 때 팩맨은 어떤 행동을 할까?

6 min 노드가 고장 났을 때(상대가 최선을 안 둘 때)도 고려하기

Minimax Efficiency

- How efficient is minimax?
  - Just like (exhaustive) DFS
  - Time: $O(b^m)$
  - Space: $O(bm)$

- Unlike ordinary search we cannot stop at the first goal. We have to look at the WHOLE tree.
  - Why? Because you never know whether the part you skipped is the part that decides the game

- For chess, b ≈ 35 and m ≈ 100.
  - An exact solution is completely impossible: $O(35^{100})$

